

Deep Learning

Assignment #2

Convolution Neural Networks

Assignment overview and motivation

- *Format: Individual or group (max 3 students) with offline defense*
- *Grading: 15 points*
- *Timeline: Assignment # 1 available in Week 7 (14th Oct), both Code submission deadline and offline defense deadline till Week 9 (28th Oct)*

While completing this assignment, you will develop and strengthen the following skills:

- *Approaching a real-world machine learning problem with given data and a target metric.*
- *Conducting effective Exploratory Data Analysis (EDA).*
- *Designing a reliable validation strategy and monitoring it throughout the entire development process.*
- *Building a CNN from scratch and using existing architectures.*
- *Performing health checks on the optimization process.*
- *Using Transfer Learning techniques and other advanced Deep Learning techniques*
- *Utilizing specialized Deep Learning frameworks and logging tools.*
- *Creating a comprehensive performance report.*

Task

Task Selection:

- Take part in the [dedicated Kaggle In-Class competition](#)

EDA and Metric Analysis:

- *Perform Exploratory Data Analysis (EDA).*
- *Analyze the proposed target metric: discuss its strengths, weaknesses, and potential edge cases.*
- *Suggest complementary metrics to capture other aspects of performance.*
- *Propose an alternative target metric, with justification.*

Validation:

- Design a validation strategy based on common sense and expected correlation with Public leaderboard scores.
- Explain your motivation for the chosen strategy.
- Discuss possible reasons for low or high correlation with leaderboard results.
- Perform adversarial validation if necessary (make sure to do it with the same model as the main task)

CNN Model:

- Set up experiment and logging infrastructure using high-level libraries (like Pytorch Lightning and W&B)
- Use existing architecture (you can find an enormous CNN zoo in [pytorch-image-models library](#))
- Experiment with different Optimizers, Schedulers, hyperparameters, and all other elements, which were covered in the Lecture materials
- Implement your own CNN OR try to add your own building block to the existing architecture. Compare the results with the previous CNN
- Collect all relevant results:
 - loss/metric curves,
 - gradient flow visualizations,
 - GradCAM visualization,
 - parameter tuning logs, etc.
- Submit predictions and compare Public leaderboard metrics with local validation results.
- Conclude whether the model is adequate.

Ensembling:

- Combine several CNN models using the ensembling (NOT stacking) technique
- [Optionally] Use Logistic Regression in order to perform stacking
- Submit predictions to obtain Public leaderboard metrics, and compare them with local validation results.

Final Report

- Prepare a comprehensive report including:
 - data insights,
 - feature engineering,
 - tried models,
 - chosen hyperparameters,
 - metrics and optimization curves,
 - model insights and conclusions.
- Reports may be prepared in LaTeX or Markdown ([README.md](#)).

NOTES

- DO NOT USE ANY ADDITIONAL OR EXTERNAL DATA in the competition — this is a RED FLAG and will result in an instant BAN! This applies to training, validation, EDA, and other possible steps. You may use any pretrained models from [pytorch-image-models library](#) and pretrained models from other libraries, but only on ImageNet.
- Do not hesitate to include failed ideas or experiments in your report. Documenting unsuccessful attempts is a strong indication that you systematically explored different approaches and gradually arrived at the best-performing solution, rather than relying on a “lucky shot.” Moreover, analyzing failed approaches can often lead to valuable insights about the dataset, the task itself,

and the behavior of deep learning algorithms. Such reflections may also highlight potential directions for future improvements and demonstrate a deeper understanding of the problem-solving process.

- Keep your EDA concise. Avoid overloading it with excessive plots, histograms, or tables. Focus only on key aspects of the data (e.g., missing values, target distribution). If a part of your EDA does not lead to meaningful conclusions, you likely don't need it (for example, full pairwise correlation matrices). Do NOT follow the typical Kaggle-style EDA – most of them are far too verbose and unfocused.
- Prepare your presentation carefully. A messy speech with too much code or too many files makes it nearly impossible for the audience to follow. Think in advance about what you want to highlight
- Using Keras, TensorFlow, or Chainer is a RED FLAG and will result in an instant BAN!
- You should thoroughly analyze the training/test split and consider time and group factors in your validation. You should also use the same splits for different ML/DL approaches so that the results can be accurately compared. And if there is a huge mismatch between CV vs kaggle LB - consider using adversarial validation.
- Your final report MUST include both CV and LB scores for all experiments.

Grading Policy and Defense

Grading Policy is outlined in the table

Criterion	Points
Overall correctness and completeness of the submitted code for the lab assignment, overall quality (metrics) of the proposed solution	8
Practical question about the submitted code during defense	2
Theoretical part during defense (2 questions + 1 exercise)	5
Criterion	Points

The defense will be conducted during a practical session, where the Lecturer will evaluate your code, report, and presentation skills. It is important to emphasize that the assessment will not be limited to metrics, analysis, and the performance of your models, but will also take into account the overall quality of your code, including readability, structure, and adherence to good practices. During the defense, the Lecturer will evaluate each student's individual contribution, demonstrated skills, and depth of knowledge. In addition, up to 2 theoretical questions may be asked; these will be based on the questions outlined in Appendix 1 and 1 Exercise from the Appendix 2, but will not necessarily be restricted to them.

AI Tools usage policy

Scope

This policy applies to all generative AI tools that produce or modify text, code, images, or other digital content. This includes, but is not limited to:

- *Text generation tools: ChatGPT, Claude, Gemini, Mistral, LLaMA-based models, etc.*
- *Code generation tools: GitHub Copilot, OpenAI Codex, TabNine, etc.*
- *Language assistance tools: Grammarly, DeepL, Google Translate, etc.*
- *Local LLMs or self-hosted AI models.*

Allowed Uses

You may use AI tools for the following purposes, provided they comply with the conditions below:

- *Spell Check & Style Correction. Example: Using Grammarly to correct grammar and improve style in a lab report or paper review.*
- *Translation. Example: Using DeepL or Google Translate to translate sections of a document into another language.*
- *Idea Generation & Pitching. Example: Brainstorming possible approaches for solving a problem or generating research ideas.*
- *Self-Education. Example: Using ChatGPT to explain an unfamiliar algorithm or concept.*
- *Library & Documentation Exploration. Example: Asking AI to summarize PyTorch documentation for a specific class or function.*
- *Code Assistance (Partial). Writing or correcting specific functions or methods (not entire solutions). Examples: Write a function that reads data and converts it to another format. Implement a data collator with padding to the longest sequence. Create an image augmentation function for training.*

Conditions for Use

Full Transparency:

- *Any use of AI tools must be disclosed in your submission.*
- *Include all prompts, responses, and/or screenshots showing your interaction with the AI tool.*
- *If using online tools, include chat links (if available) or full transcripts in an appendix of your report.*

Partial Contribution Only:

- *AI tools should support your work – they must not produce the complete assignment, lab, or report.*
- *You must be able to explain and defend any AI-generated content during oral defense or Q&A*

Examples

Acceptable Uses:

- *"Correct grammar and style in this paragraph of my report."*
- *"Write a function to load images from a directory and resize them."*
- *"Suggest 3 ideas for improving model generalization."*

Unacceptable Uses:

- *"Write the entire peer review for this paper."*
- *"Generate the complete solution code for this assignment task."*

Additional Points Opportunity

At the end of November, the [dedicated Kaggle In-Class competition](#) will conclude, and Private scores will be released. Based on them 2 best teams (**ONLY IF the team outperforms effnetb0_baseline**) will get:

1. Each member of the first team - 5 points
2. Each member of the second team - 3 points

Appendix 1 (Theoretical Questions)

- What are the advantages of representing an image in RGB vs. HSV vs. LAB color spaces for computer vision tasks?
- Why is convolution preferred over fully connected layers for image data?
- Explain how the receptive field grows with depth in a CNN. Why is this property important?
- What is an impulse image, and how does it relate to the concept of an impulse response in control theory?
- Why does weight sharing in CNNs act as a form of regularization?
- Compare the roles of convolution, pooling, and normalization layers in a CNN.
- How does Batch Normalization improve convergence and stability during training?
- What kind of inductive biases are built into CNNs? Why are they beneficial for image-based tasks?
- Why might one choose average pooling over max pooling (and vice versa)?
- How would the output of a convolution change if the kernel size were doubled but the stride remained constant?
- Describe how padding affects both the receptive field and output dimensions of a CNN layer.
- Why is a ReLU activation function typically used after convolution layers instead of sigmoid or tanh?
- Suppose we remove the bias term from all CNN layers. How would it impact model capacity?
- How does adding BatchNorm between convolution and activation affect the network's representational power?
- Why is convolution commutative?
- How can we express a Fully Connected layer through Convolution and vice versa?
- Give a definition of CNN?
- What are the main limitations of AlexNet?
- What problem do residual connections in ResNets aim to solve?
- How does the gradient flow through a residual block differ from that in a plain CNN block?
- What are the main dimensions of scaling in CNNs, and how does EfficientNet jointly optimize them?
- Explain the compound scaling coefficient in EfficientNet. Why is balancing width, depth, and resolution better than scaling one dimension independently?
- Discuss how transfer learning leverages pre-trained CNNs. Why does it work so well even across domains (e.g., ImageNet → medical imaging)?
- What are the potential pitfalls of naive transfer learning (e.g., freezing too many or too few layers)?
- How would you design an augmentation pipeline for fine-grained classification tasks (e.g., bird species) vs. coarse ones (e.g., animal vs. vehicle)?
- How should we perform validation for a stacking - 2-level model?
- Explain how BatchNorm works during training and inference.

- *Why might models with residual connections generalize better even if they have more parameters?*
- *Compare ResNet and EfficientNet in terms of training efficiency, interpretability, and deployment cost.*
- *What are the main differences between digital waveform and spectral representations of sound?*
- *Why are CNNs effective for processing spectrograms?*
- *Explain how 1D and 2D convolutions differ when applied to audio data.*

- *How do time-frequency trade-offs (e.g., window size in STFT) affect CNN learning in spectrogram-based models?*
- *Why might mel-spectrograms or log-mel features be preferable to raw spectrograms?*
- *What is the intuition behind using pre-trained CNNs on spectrograms?*
- *How does data augmentation differ between image and audio domains? Give examples.*
- *What are typical sources of noise or domain shift in audio datasets, and how can CNNs be made robust to them?*
- *Compare the inductive biases of CNNs in the context of audio classification.*
- *How can CNN architectures be adapted for real-time or low-latency audio inference?*
- *Explain why normalization and pooling strategies are especially important in audio models.*
- *Discuss how the receptive field relates to temporal context in audio classification.*
- *How might you modify CNN architectures for tasks like sound event localization or multi-label audio tagging?*

Appendix 2 (Practical Exercises)

1. 2-D Convolution Computation

Let the input feature map

$$X = \begin{bmatrix} 1 & 2 & 0 \\ -1 & 0 & 1 \\ 2 & 3 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}, \quad s = 1, p = 0.$$

- (a) Compute the convolution output $Y = X * K$ using valid padding.
- (b) Now assume zero-padding $p = 1$ and stride $s = 2$. Compute only the resulting output **shape** (no numerical computation).

2. CNN Layer Resolution Propagation

A 128×128 grayscale image passes through the following layers:

Layer	Kernel	Stride	Padding
Conv1	3×3 , 32 filters	1	1
MaxPool1	2×2	2	0
Conv2	5×5 , 64 filters	1	0
MaxPool2	2×2	2	0

- (a) Compute the spatial resolution after each layer.
- (b) Compute the total number of trainable parameters (ignore biases).

3. Receptive Field Calculation

A CNN has the following layers:

Layer	Kernel Size (k)	Stride (s)
Conv1	3	1
MaxPool1	2	2
Conv2	3	1
Conv3	3	1

- (a) Derive the receptive field R_i for each layer i .

4. Gradient Through a Residual Block

Residual block output:

$$y = f(x, W) + x, \quad \text{where } f(x, W) = \text{ReLU}(Wx).$$

- (a) Derive $\frac{\partial L}{\partial x}$ given the upstream gradient $\frac{\partial L}{\partial y}$.
- (b) Explain why residual connections help prevent the gradient vanishing problem in deep CNNs.